Инженерные практики

Что именно проверяется автоматически и как это запустить. Всё ниже — факты репозитория на дату сдачи, без обещаний.

Что гоняет CI

Файл `.github/workflows/ci.yml`. Каждый PR и каждый пуш в `main` запускают семь независимых заданий:

Задание	Что делает
`backend-lint`	`ruff check` (правила E, F, I, B, UP); `mypy --strict` на ядрах (список в `pyproject.toml`, раздел «Типы»); `mypy` по всему `app` для информации (долг считаем числом)
`backend-tests`	`pytest` на PostgreSQL 16 + pgvector 0.8.7 (service container), три шарда: `api`, `engine` (антиподсказчик, банк, CAT, QA, таксономия), `rest`. Node 22 нужен JS-семействам банка заданий. Вывод `--durations=15` показывает самые долгие тесты
`migrations`	`alembic upgrade head`, затем `downgrade base`, снова `upgrade head`, затем `alembic check`: модели и миграции совпадают, неописанных изменений схемы нет
`frontend`	`npm ci`, `tsc -b --noEmit`, `eslint`, `vite build`, `npm audit --omit=dev --audit-level=high`
`e2e`	поднимает БД, пул заданий, демо-мир, бэкенд (DEMO_MODE) и Vite, гонит Playwright, выкладывает отчёт и логи артефактом
`docker`	сборка образов `backend` (он же `pool`), `web` (nginx + интерфейс), `fsp-mock`; кэш GitHub Actions
`dependency-audit`	`pip-audit` по прод-зависимостям из `uv.lock`

Файл проверен `actionlint` (замечаний нет). Установленные версии зависимостей фиксируют `uv.lock` и `package-lock.json`; CI ставит их строго по локам (`UV_FROZEN`, `npm ci`).

Тесты

`cd backend && uv run pytest -q` — 2297 тестов (в последнем полном прогоне 2276 прошли, 21 пропущен, 0 упало). Полный прогон занимает около 22 минут на ноутбуке разработчика, поэтому CI делит набор на три шарда. Распределение по областям (по префиксу файлов):

Область	Тестов	Что покрыто
Банк заданий (`test_bank_*`)	698	контракт семейств, генераторы, эталонные решения, гейты качества
HTTP API (`test_api_*`)	294	воронка кандидата и работодателя, приглашения, чат, ATS, демо-контур
Подбор (`test_matching_*`)	248	разбор брифа, Score, объяснения, версии подборки, покрытие
Антиподсказчик (`test_anticheat_*`)	240	двойники, улики, песочница жюри, пороги ложных срабатываний
Контакты (`test_contact_*`)	199	жизненный цикл приглашения, таймеры, квоты, приватность
ФСП (`test_fsp_*`)	109	модель достижений, VSS, адаптер, мок реестра
Платформа (`test_platform_*`)	109	аутентификация, пароли, безопасность, схема OpenAPI, модели
IRT/CAT (`test_cat_*`)	99	движок, калибровка, детекторы, экспресс-режим, сессии
Оценка на синтетике (`test_eval_*`)	52	популяция с латентной истиной, метрики, спецификация валидации
Матрица доступа (`test_security_matrix.py`)	14	см. раздел «Безопасность»

Остальное — тестовые файлы по резюме, PDF, планировщику, LLM-границам, массовым запросам и т. п. Контрольные примеры из спецификаций оформлены обязательными тестами. Каждый тест с БД работает в откатываемой транзакции в отдельной базе на запуск (параллельные запуски не мешают друг другу), а время в тестах задаёт замороженные часы `frozen_clock`.

Сквозные тесты (Playwright)

Каталог `frontend/e2e` — отдельный пакет, чтобы не добавлять зависимости в основной `package.json` и не попадать в образ интерфейса. `@playwright/test` распространяется по лицензии Apache-2.0. Пять сценариев идут по порядку номеров, потому что второй и четвёртый опираются на данные предыдущих:

1. `01-jury-express`: `/jury`, экспресс-тест, ответы на задания, экран результата.
2. `02-candidate-journey`: регистрация (в DEMO_MODE аккаунт активен без письма), опрос из четырёх шагов, выбор грейда, старт теста.
3. `03-employer-flow`: вход работодателя, бриф с вилкой, подборка, панель доказательств (ровно три «за» и один «против»), отправка приглашения.
4. `04-mutual-chat`: кандидат принимает приглашение, даёт согласие на раскрытие контактов, видит «Взаимно» и контакт компании, пишет в чат.
5. `05-sandbox-twin`: `/try`, копирование условия подменяется двойником (в буфере другое значение, счётчик «копировали»), итог сверки.

Запуск: `make e2e` (нужна БД с демо-миром: `make up-infra migrate gen-pool seed`). Скрипт `frontend/e2e/run.mjs` сам поднимает бэкенд на :8128 и Vite на :5185 и гасит их после прогона. Перед прогоном `global-setup.ts` сбрасывает демо-мир через `POST /api/demo/reset`, поэтому тесты повторяемы. Логины демо-аккаунтов берутся из `backend/var/demo_accounts.md` (в git не попадает). Весь набор также проверен на production-сборке `vite build` с политикой CSP из `frontend/csp.conf`: нарушений политики нет.

Ограничение: лимит попыток входа (5 за 5 минут на email) учитывается и в e2e. На одном и том же бэкенде более пяти прогонов подряд за пять минут упрутся в него; перезапуск бэкенда сбрасывает счётчики.

Типы

`mypy` стоит в dev-группе (лицензия MIT).

- `uv run python scripts/typecheck_strict.py` (или `make typecheck`) запускает `mypy --strict` на 25 файлах ядер: IRT/CAT-движок и сессии (`app/testing`), скоринг, объяснения и подборка (`app/matching`), VSS и модели ФСП, ядро безопасности (`app/core`: пароли и токены, лимитер, доступ, аудит, часы, конфиги, согласия). Список лежит в `pyproject.toml`, раздел `[tool.vzaimno.typecheck_strict]`. Ошибок нет, в CI гейт блокирующий.
- `uv run mypy` проверяет весь `app` с умеренными настройками. Было 344 ошибки на 62 файлах, после правок осталось 258 (на момент сдачи 290: за это время в других ветках работ добавился код). Остаток — в банке заданий, демо-сервисах, разборе брифа и сервисных слоях; типовые причины: повторное использование имени переменной с другим типом, `Optional` без проверки, `Result.rowcount` у SQLAlchemy. В CI этот шаг информативный: число ошибок видно в логе, но сборку он не роняет.

Статический анализ и стиль

`ruff` (E, F, I, B, UP, длина строки 110) без замечаний. Фронтенд: `tsc -b --noEmit` и `eslint` (typescript-eslint, react-hooks, react-refresh) без замечаний.

Безопасность

Проверено на уровне OWASP Top-10, коротко:

- A01, контроль доступа. Точечные проверки чужих ресурсов есть в `test_api_chat`, `test_api_ats`, `test_api_needs_privacy`, `test_api_contact_*` и `test_api_testing_integrity`. Сквозная матрица `tests/test_security_matrix.py` добавлена в рамках аудита: (1) по схеме OpenAPI каждый защищённый маршрут без токена не отвечает 2xx и 5xx; (2) работодатель другой компании не читает и не меняет чужое приглашение и чужой чат; (3) чужой кандидат не отвечает на приглашение, не открывает чужую сессию теста, не шлёт события и ответы; (4) границы ролей (кандидат, работодатель, методист, администратор). Уязвимых мест не найдено.
- A02, криптография. Argon2id (время 3, память 64 МиБ, параллелизм 4); JWT с явным списком алгоритмов при проверке; короткий access (15 минут) и refresh-токен в `HttpOnly`-cookie с ротацией и защитой от повторного использования; в `VZ_ENV=prod` запрещены секреты по умолчанию и короткие секреты.
- A03, инъекции. Сырой SQL один в коде приложения с параметрами; единственная сборка условия из строки — `postgresql_where` индекса из констант-перечислений статусов. Параметризация везде, где значения приходят извне.
- A04/A05, конфигурация. Лимиты запросов на вход, регистрацию, подтверждение, повторную отправку и обновление токена (`VZ_RL_*`); CORS только из явного списка origin; заголовки `X-Content-Type-Options`, `X-Frame-Options`, `Referrer-Policy` в бэкенде и в nginx; HSTS в Caddy (прод). Добавлено в рамках аудита: `Content-Security-Policy` для статики интерфейса (`frontend/csp.conf`), политика проверена полным e2e-прогоном.
- A06, зависимости. `pip-audit` по прод-зависимостям бэкенда: уязвимостей нет. `npm audit --omit=dev`: уязвимостей нет. Полный `npm audit` показывает уязвимости только в инструментах сборки (`sharp`, `braces`, `postcss-selector-parser`); в образ интерфейса они не попадают.
- A07, аутентификация. Подтверждение email кодом с ограничением попыток и частоты отправки; ограничение попыток входа по email и по IP.
- A08/A09, целостность и журналирование. Вебхуки ATS подписаны HMAC-SHA256 и идемпотентны; журнал аудита ключевых действий.
- Секреты в репозитории. Поиск по шаблонам ключей и токенов в отслеживаемых файлах: реальных секретов нет. Демонстрационные значения лежат в `services/fsp-mock/fsp_mock/keycloak.py`, `services/fsp-mock/fsp_mock/main.py` и `deploy/keycloak/realm-fsp.json` и помечены как демо; `.env` и `backend/var/` в `.gitignore`.

Что сознательно не закрыто: `Strict-Transport-Security` отдаёт только Caddy (боевой стенд за ним), локальный nginx работает по HTTP; автоматический динамический скан (OWASP ZAP) не запускалась.

Детерминизм и конфигурация

- Случайность только через явный `random.Random(seed)` или `numpy.random.default_rng(seed)`; глобального `random` нет. Секретные seed экземпляров заданий получаются HMAC от `VZ_APP_SECRET`.
- Все веса, пороги, квоты и справочники лежат в `config/*.yaml` (14 файлов, около 3000 строк). В коде магических чисел нет; настройки инфраструктуры и безопасности — переменные `VZ_*` (`.env.example`).
- LLM не вызывается в рантайме теста и ранжирования; проверка границы — `tests/test_llm_boundaries.py`.
- Резюме и самоописание не влияют на категорию и Score, отсутствие истории ФСП не штрафуется, контакты видны работодателю только после раскрытия (разные DTO `CandidateBlind` и `CandidateDisclosed`): каждое правило закреплено тестами.

Миграции

Alembic, 13 ревизий (`backend/alembic/versions`). Цикл `upgrade, downgrade, upgrade` и отсутствие дрейфа между моделями и схемой проверяются в CI. В `alembic/env.py` добавлен импорт демо-моделей (`app.services.demo.models`): без него `alembic check` считал бы демо-таблицы лишними.